

**LABORATORIO ARCHITETTURA DEGLI
ELABORATI - ELABORATO ASSEMBLY x86 AT&T**

Lorenzo Lugli, Riccardo Ueberbacher

Università degli studi Verona



Software per la pianificazione delle attività di un sistema produttivo

Per un massimo di 10 prodotti in 100 unità di tempo dette “slot temporali” il programma organizza i prodotti in modo che il sistema produttivo produca i prodotti nell'ordine determinato dall'utente. Durante ogni slot temporale il sistema può produrre un solo prodotto alla volta.

Ogni oggetto è caratterizzato da quattro elementi:

- Codice identificativo da 1 a 127.
- Durata del processo di produzione da 1 a 10 slot.
- Scadenza: tempo massimo per la produzione da 1 a 100 slot.
- Priorità da 1: minima a 5: massima.

I prodotti e i relativi dati vengono letti dal file specificato negli argomenti CLI.

Il software permette di organizzare la produzione utilizzando due algoritmi di pianificazione, per una gestione ottimale degli slot temporali.

L'utente può scegliere tra i 2 algoritmi per pianificare la produzione:

1. EDF: Earliest Deadline First: il prodotto con la più breve scadenza viene prodotto per primo.
2. HPF: Highest Priority First: il prodotto con la priorità più alta viene prodotto per primo.

Una volta selezionato l'algoritmo, il programma elabora un piano di produzione che indica l'ordine in cui i prodotti devono essere lavorati. Alla fine, il software visualizza:

- L'ordine in cui iniziano i prodotti,
- L'unità di tempo in cui la produzione termina,
- Le penalità totali, se ci sono ritardi rispetto alle scadenze.

La penalità è il costo che l'azienda paga per i ritardi, calcolato moltiplicando le unità di tempo di ritardo per la priorità del prodotto.

File principali

main.s

Acquisisce il path dei file da leggere, e opzionalmente su cui scrivere, da linea di comando se gli argomenti sono errati stampa un errore specifico in std-err. Inserisce poi i dati in un array¹. Richiede all'utente quale algoritmo utilizzare, stampa il messaggio di conferma e chiama la funzione del relativo algoritmo.

fileIO.s

Contiene funzioni utilizzate nel programma per l'apertura e chiusura dei file sia in lettura che in scrittura. Inoltre contiene anche la funzione di lettura del file in ingresso.

algo.s

Riceve in input l'array¹ contenente i prodotti calcola il tempo di produzione con l'algoritmo selezionato e l'eventuale penalità stampando poi oltre che in std-out anche su file se aperto.

bubbleSort.s

Ordina l'array¹ utilizzando un algoritmo Bubble sort. L'array¹ viene fornito nei parametri della chiamata di funzione

print.s

Contiene funzioni per la stampa a std-out a std-err e se è presente un file aperto in scrittura può scrivere anche su di esso.

Struttura dei dati

I dati sono memorizzati in un array¹ monodimensionale, che al interno del codice viene trattato come un array¹ di al massimo 10 oggetti ognuno dei quali contiene 4 elementi: Identificativo, Durata, Scadenza, Priorità.

Ogni elemento viene memorizzato in un singolo Byte il quale può rappresentare fino a un intero con segno compreso tra -128 e +127 o fino a un intero senza segno compreso tra 0 e 255. Dato che l'intero più grande che può essere contenuto nel file .txt in ingresso è 127 un Byte è sufficiente per memorizzarlo.

$$4 \text{ Byte}(1 \text{ oggetto}) \times 10(\text{oggetti max}) = 40 \text{ Byte}$$

Quindi la dimensione totale della struttura dati con massimo 10 prodotti/oggetti occupa 40 Byte di spazio nella sezione .bss (ovvero la HEAP) del programma. I valori vengono tutti inizializzati a 0 e poi popolati in base al file in ingresso. Grazie a questa struttura dati basta leggere una sola volta il file in ingresso al inizio dell'esecuzione. Questo rende il programma più efficiente se si prevede di eseguire entrambi gli algoritmi dato che i valori del file sono già memorizzati. Non è necessario leggere il file in ingresso ogni volta che viene eseguito un algoritmo. Quindi il programma risulta più veloce.

Struttura del codice e Funzioni

main.s

_start:

Questa funzione iniziale controlla se è stato fornito il file da cui leggere. Se non viene specificato un file, viene mostrato un messaggio di errore. Inoltre controlla se è stato fornito un file su cui scrivere l'output del programma, se è presente viene passato alla funzione openFile in fileIO.s per l'apertura in scrittura.

_readFileIO:

Passa alla funzione openFile il nome del file da leggere, openFile ritorna il file descriptor che viene usato poi dalla funzione readFile che si occupa di popolare l'array¹ con tutti i dati presenti nel file. Infine il file viene chiuso con la funzione closeFile.

_mainMENU:

Presenta all'utente il menu principale chiedendo quale algoritmo utilizzare:

1. EDF salta all'etichetta _EDF
2. HPF salta all'etichetta _HPF
3. EXIT salta all'etichetta _exit

legge l'input fornito per saltare all'etichetta corrispondente. Questa sezione di codice pretende uno di questi input. Finché non lo ottiene non avanza al resto del codice.

_exit:

Se c'è un file aperto in scrittura lo chiude ed esce dal programma correttamente.

_HPF:

Chiama la funzione printSTR che visualizza il messaggio "Pianificazione HPF:" e chiama la funzione ALGO con l'indicazione di ordinare per priorità ovvero
sortingID==3

_EDF:

Chiama la funzione printSTR che visualizza il messaggio "Pianificazione EDF:" e chiama la funzione ALGO con l'indicazione di ordinare per scadenza ovvero
sortingID==2

_noArgsExit:

Chiama la funzione printERR che visualizza il messaggio "ERRORE: specificare un filename come argomento." e salta all'etichetta _exit.

fileIO.s***Funzione openFile:***

Apre il file dato come parametro nella modalità specificata da un altro parametro (lettura o scrittura) e controlla se il file è stato aperto correttamente. In caso ci sia un errore salta all'etichetta `_invalidFilename`.

_invalidFilename:

Chiama la funzione `printERR` che visualizza "ERROE: si assicuri che il filename specificato esista." e salta all'etichetta `_exitERROR`.

_exitERROR:

Esce dal programma restituendo 1, esecuzione fallita.

Funzione closeFile:

Chiude il file descriptor che viene fornito come parametro.

Funzione readFile:

Prende come parametri l'indirizzo base del array¹ e il file descriptor di un file, aperto in lettura. Legge un carattere alla volta. Quando incontra il carattere EOF, ovvero la fine del file, salta a `_checkVals`. Poi controlla se il carattere letto è CR in tal caso va al prossimo carattere, se non è CR controlla se è LF o “,” in tal caso ha acquisito tutto il valore e salta a `_storeTemp`. Altrimenti procede con il controllo NAN(Not A Number), se il valore acquisito non è un intero allora salta all'etichetta `_NAN`.

A questo punto converte il carattere in intero, e lo somma a quello che è presente in `%al`, ovvero l'intero acquisito fino a ora. Se il valore in `%al` con questa somma supera 255 allora l'esecuzione salta all'etichetta `_overflow`.

_storeTemp:

Memorizza il valore in `%al`, che rappresenta l'intero memorizzato nel array¹ all'indirizzo base + `%edi`. `%edi` viene usato come contatore per allocare

correttamente i valori nelle loro corrispondenti posizioni in base al file in ingresso. Infine azzera il registro %al per il prossimo valore e incrementa %edi (la posizione nel array¹).

_closeFileExit:

Chiude il file e salta all'etichetta _exitERROR.

_overflow:

Chiama la funzione printERR che visualizza "ERRORE: overflow rilevato, si assicuri che i valori e la formattazione del file in input rispetti le specifiche del progetto." e salta all'etichetta _closeFileExit.

_NAN:

Chiama la funzione printERR che visualizza "ERRORE: uno dei valori all'interno del file non è un numero." e salta all'etichetta _closeFileExit.

_missingEOF:

Chiama la funzione printERR che visualizza "ERRORE: end of file non alla fine di una nuova linea. Per favore inserisca una nuova linea vuota alla fine del file\n Oppure una delle righe nel file contiene più o meno di 3 separatori." e salta a _closeFileExit.

_outOfRange:

Chiama la funzione printERR e printINT per visualizzare "ERRORE: il valore 'N' non rientra nelle specifiche del progetto.", dove N è %al quindi l'elemento in questione e salta a _closeFileExit.

_checkVals:

Controlla se il numero totale di valori (totalVals) memorizzati è divisibile per 4. Se non lo è qualcosa è andato storto, probabilmente nel file manca una riga vuota e salta a _missingEOF. Prepara i registri per _checkValsLoop.

_checkValsLoop:

Scrolla l'array¹ di valori acquisiti e controlla che rispettino le specifiche del progetto come ad esempio il campo Priorità che deve essere compreso tra 1 e 5. Se uno dei valori non rispetta le specifiche salta a _outOfRange. Alla fine, quindi se tutto è nelle specifiche ritorna dalla funzione readFile.

algo.s***Funzione ALGO:***

Chiama la funzione bubbleSort con l'indirizzo base del array¹, l'elemento in base a cui ordinare l'array¹ e il numero totale di oggetti all'interno dell array¹. Tutto questo viene passato a sua volta come parametro alla funzione ALGO da main.s. Inoltre viene anche passato il file descriptor del possibile file aperto.

In base al sortingID dato quindi "3" per HPF o "2" per EDF questa funzione si comporta diversamente perché nel caso HPF la funzione scrolla, array¹ ordinato al contrario dato che 5 corrisponde alla priorità massima e 1 alla minima; bubbleSort ordina da più piccolo a più grande.

Tenendo a mente questo l'esecuzione arriva all'inizio del loop.

_nextObject:

Controlla se abbiamo finito gli oggetti guardando la variabile totalObjects che viene decrementata alla fine di questa etichetta. Se abbiamo finito salta a _ALGODone.

Poi controlla se c'è bisogno di calcolare la penalità ovvero se scadenza > currentTime. Se c'è bisogno di calcolare la penalità viene chiamata la funzione calcPenalty altrimenti si procede a _calcNextObject.

_calcNextObject:

Avanza l'indirizzo base dell'array di 4 byte andando così al prossimo oggetto.

Chiaramente, come spiegato in precedenza se siamo in esecuzione HPF allora viene diminuito di 4 byte per scorrere dalla fine al inizio.

_newObject:

Salva la durata di produzione del oggetto in una variabile temporanea per decrementarla quando viene incrementata currentTime. Poi chiamando le funzioni printSTR e printINT più volte visualizza il messaggio "ID:currentTime\n". Prima di farlo salva %eax(che contiene la penalità) nello stack.

_timeLoop:

Questo è il loop principale della funzione che si occupa di incrementare il tempo(currentTime) e simultaneamente decrementare la durata del oggetto che sta producendo. Controlla inoltre se siamo arrivati al tempo massimo(100), in tal caso salta a _ALGODone. Inoltre controlla se la durata di un oggetto arriva a 0 salta all'etichetta _nextObject. In caso non sia stato fermato salta a _timeLoop.

_ALGODone:

Controlla se l'ultimo oggetto ha una penalità da calcolare e in caso chiama calcPenalty. Altrimenti procede a _ALGOret.

_ALGOret:

Chiamando le funzioni printSTR e printINT più volte visualizza:

“Conclusion: N

Penalty: M ”

Dove N corrisponde al tempo di conclusione del programma e M alla penalità con cui si è concluso. Alla fine ritorna.

Funzione calcPenaly:

Calcola la penalità e la somma a quella precedente, memorizzata in %eax, con questa formula:

$$\text{Penalità} = (\text{currentTime} - \text{deadline}) * \text{priorità} + \text{Penalità}$$

Questa funzione viene chiamata in mezzo all'esecuzione del algoritmo quindi prima di cambiare i valori nei registri li salva per poi re impostarli.

bubbleSort.s***Funzione bubbleSort:***

Applica l'algoritmo conosciuto "Bubble sort" che ordina l'array¹ dato dal valore più piccolo al valore più grande di uno degli elementi degli oggetti. L'elemento scelto anche chiamato sortingID viene passato come parametro insieme all'indirizzo base dell'array¹ e il numero totale di oggetti presenti nel array¹. Anche questa funzione salva tutti i registri perché viene chiamata da ALGO a cui servono i valori che ha nei registri.

print.s***Funzione strlen:***

Calcola la lunghezza di una stringa in %ecx scorrendola finché trova "/0". Ritorna la lunghezza della stringa.

Funzione printSTR:

Chiama strlen sulla stringa ottenuta come parametro, poi stampa la stringa in std-out o file descriptor in base al parametro dato. Salva i registri perché può essere chiamata a qualsiasi punto senza cambiare i registri del caller.

Funzione printINT:

Converte l'intero(in %eax) in un argomento alla volta e poi lo stampa tutto insieme in std-out o file descriptor in base al parametro dato. Salva i registri perché può essere chiamata a qualsiasi punto senza cambiare i registri del caller.

Funzione printERR:

Chiama strlen sulla stringa ottenuta come parametro, poi stampa la stringa in std-err. Salva i registri perché può essere chiamata a qualsiasi punto senza cambiare i registri del caller.

Funzione printWRITESTR:

Controlla se il file descriptor, passato come parametro, è nullo. In tal caso salta a _printSTR. In caso contrario vuol dire che c'è un file aperto in modalità di scrittura quindi procedere a stampare sul file chiamando printSTR e passando come parametro il file descriptor e la stringa. Salva i registri perché può essere chiamata a qualsiasi punto senza cambiare i registri del caller.

_printSTR:

Chiama printSTR e passa come parametro std-out e la stringa, quindi visualizza un messaggio in CLI.

Funzione printWRITESTR:

Controlla se il file descriptor, passato come parametro, è nullo. In tal caso salta a _printINT. In caso contrario vuol dire che c'è un file aperto in modalità di scrittura quindi procedere a stampare sul file chiamando printINT e passando come parametro il file descriptor e l'intero in %eax. Prima di stampare salva %eax per non modificarlo. Salva i registri perché può essere chiamata a qualsiasi punto senza cambiare i registri del caller.

printINT:

Chiama printINT e passa come parametro std-out e l'intero in %eax, quindi visualizza un messaggio in CLI.

TESTING

Il processo di testing è avvenuto attraverso la generazione manuale di file di input che generassero un output prevedibili

EDF.txt

Produce un output senza penalità con l'algoritmo earliest deadline first invece crea una penalità di 6 se si usa l'algoritmo highest priority first.

Both.txt

Entrambi gli algoritmi non creano una penalità e producono entrambi lo stesso risultato.

NAN.txt

Viene usato per testare il controllo che i valori nel file siano dei numeri.

None.txt

Ne HPF ne EDF riescono a trovare una soluzione senza penalità.

overflow.txt

Testa il controllo di un overflow cioè che uno dei valori sia sopra il massimo consentito o che i prodotti siano più del massimo consentito oppure che i valori di ogni prodotto siano più di 4.

wrongDeadline.txt/wrongID.txt/wrongPrio.txt/wrongDuration.txt/wrongInput.txt

Testano rispettivamente i controlli dei valori specificati del nome e del NAN.

SCELTE PROGETTUALI

Se nel file di input manca un valore cioè è presente n,n,,n l'assenza del valore viene interpretato come uno 0 quindi il prodotto viene interpretato come n,n,0,n.

Il file in input viene aperto una sola volta quindi anche se si chiama l'algoritmo più volte lavora sempre sugli stessi dati anche se vengono modificati o rimossi. Questo lo rende più veloce.

Array¹: L'array non esiste come tipo di dato su Assembly quindi in realtà questo è semplicemente uno spazio riempito di zeri costituito da 40Byte che viene navigato a partire da un indirizzo base(ordiniArr) e sommandoci contatori o spostando l'indirizzo base. Possiamo anche vedere questo spazio come un insieme di struct tutti uno di fianco all'altro in memoria. Essenzialmente abbiamo creato una struttura dati che si comporta come gli array in C senza le parentesi quadre.